

Cloud Provider Analytics

ETL + Streaming + Serving en Cassandra

Proyecto Final - Big Data

Segundo Cuatrimestre 2025

Autores:

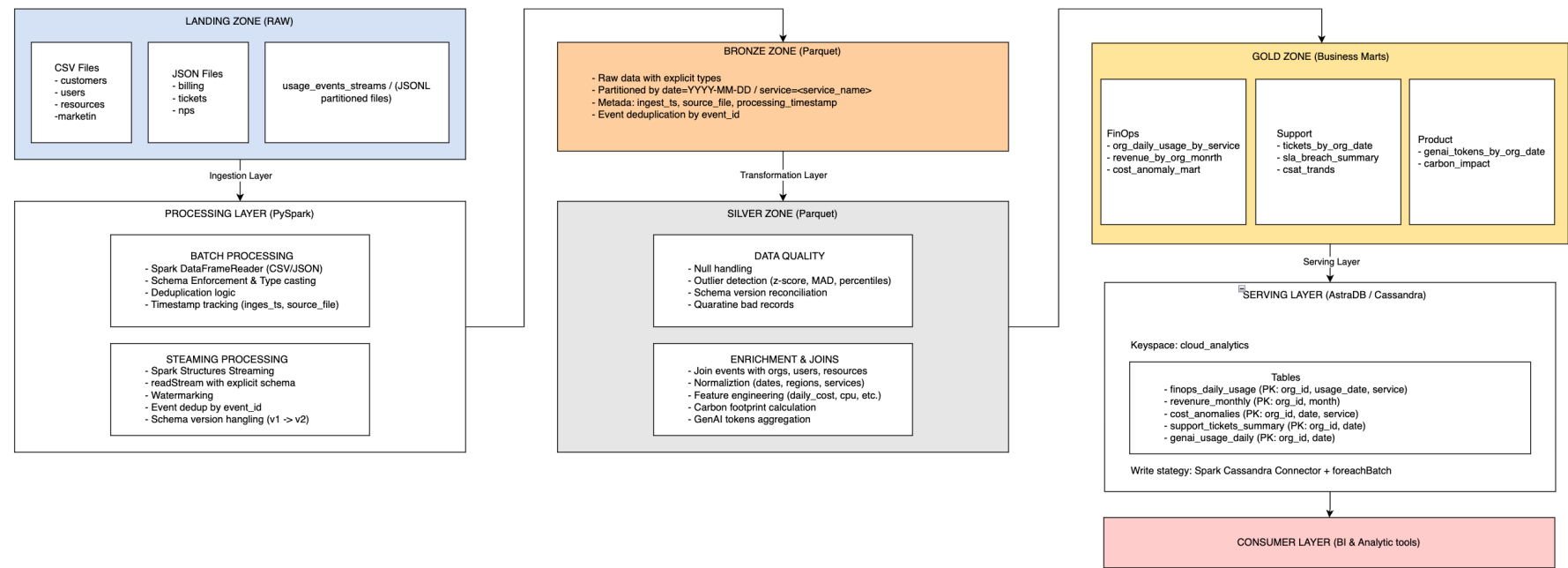
Nicolás Rossi & Juan Ramiro Castro

Índice

- 1. Arquitectura y Diseño**
- 2. Stack Tecnológico**
- 3. Zonas del Data Lake**
- 4. Patrón Arquitectónico (Lambda)**
- 5. Implementación del Pipeline**
- 6. Calidad de Datos (MAD)**
- 7. Serving Layer: Cassandra**
- 8. Demos & Consultas**

1. Arquitectura y Diseño

Arquitectura de Alto Nivel



Componentes

Capas

- **Ingestion:** Batch (CSV) + Streaming (JSONL).
- **Processing:** PySpark 3.5.0.
- **Storage:** Parquet en Google Drive.
- **Serving:** AstraDB (Cassandra-aaS).

Zonas del Data Lake

- **Landing:** Raw inmutable.
- **Bronze:** Tipificación + metadata.
- **Silver:** Limpieza + enriquecimiento.
- **Gold:** Marts analíticos.
- **Quarantine:** Registros rechazados.

2. Recursos y Tecnologías

Stack Tecnológico

Procesamiento y Storage

- **Compute:** Google Colab (Python 3.10).
- **Engine:** PySpark 3.5.0.
- **Storage:** Google Drive (Parquet/Snappy).
- **Detección:** MAD (Median Absolute Deviation).

Base de Datos

- **AstraDB (Free Tier):** 40GB storage.
- **Connector:** spark-cassandra-connector.
- **Auth:** Client ID/Secret.

3. Zonas del Data Lake

Estructura del Data Lake

Zona	Propósito	Particionamiento	Características
Landing	Raw inmutable	Por fuente	Origen de verdad.
Bronze	Tipificado + Metadata	date=YYYY-MM-DD	Schema explícito, deduplicación.
Silver	Conformado + Enriquecido	date=YYYY-MM-DD	Joins, normalización, quality checks.
Gold	Marts analíticos	Por caso de uso	Agregaciones, KPIs de negocio.
Quarantine	Rechazados	date , error_type	Para análisis y reingesta.

Datasets Principales

Batch (Maestros)

- **Clientes:** `customers_orgs.csv`, `users.csv`.
- **Operaciones:** `resources.csv`, `support_tickets.csv`.
- **Finanzas/Mkt:** `billing_monthly.csv`, `marketing_touches.csv`, `nps.csv`.

Streaming (Eventos)

- **Fuente:** `usage_events_stream/*.jsonl` (~100 archivos).
- **Span:** ~60 días.
- **Evolución:** Cambio de esquema v1 → v2 detectado hace ~45 días.

4. Patrón Arquitectónico: Lambda

¿Por qué Lambda?

Batch Layer

- Datos maestros y facturación.
- Frecuencia diaria/mensual.
- Mayor eficiencia en grandes volúmenes históricos.

Speed Layer

- Eventos de uso (`usage_events`).
- Near real-time.
- Detección inmediata de anomalías.

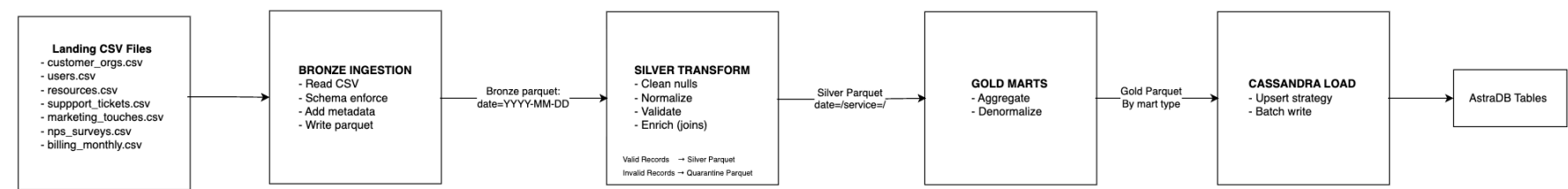
Decisión vs. Kappa

Se descartó **Kappa** porque "restreamear" datos estáticos (CSVs maestros) añade complejidad de estado y costo operativo sin aportar valor al negocio.

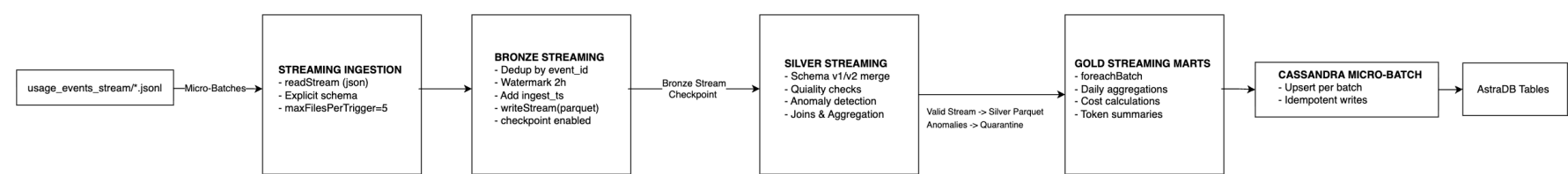
5. Implementación del Pipeline

Batch & Streaming Flows

Batch



Streaming



Transformaciones (Silver)

Transformación	Detalle Técnico / Lógica Implementada
Reconciliación Schema	Unificación <code>events</code> v1/v2. Schema superset: campos nuevos (<code>genai_tokens</code>) → <code>NULL</code> en v1.
Manejo Temporal	<code>to_timestamp()</code> para eventos y extracción de <code>usage_date</code> (<code>to_date</code>) para particionamiento.
Validación Costos	Filtro de calidad <code>cost_usd >= -0.01</code> (permite ajustes contables negativos mínimos).
Enriquecimiento	<code>LEFT JOIN</code> con dimensiones Bronze: Orgs (plan, industria), Users (rol) y Resources .
Normalización	<code>lower(trim())</code> aplicado a <code>service</code> y <code>region</code> para consistencia.
KPIs Derivados	Métrica <code>avg_tokens_per_request</code> : división segura usando <code>NULLIF(requests, 0)</code> .

Marts Analíticos (Gold)

Mart	Agregación / Lógica
finops_daily_usage	Agrupado por servicio/día. SUM(cost) , COUNT(reqs) + Detección Anomalías (MAD).
org_service_cost_summary	Ventanas (1d, 7d, 14d). Ranking de servicios por costo acumulado (dense_rank).
support_tickets_summary	Agrupado por severidad. KPIs: COUNT(tickets) , AVG(csat) , COUNT(sla_breach) .
revenue_monthly	Por mes de facturación. Neto: (subtotal + taxes - credits) * FX .
genai_usage_daily	Filtro service='genai' . KPIs: SUM(tokens) , COUNT(reqs) , AVG(tokens/req) .
carbon_footprint_daily	Filtro carbon_kg > 0 . SUM(carbon_kg) y conteo de eventos (Sostenibilidad).

6. Calidad de Datos

Estrategia de Calidad y Anomalías (MAD)

Reglas

1. **Schema Enforcement:** Validación estricta de tipos, evita la inferencia de Spark.
2. **Deduplicación:** Por `event_id` en streaming.
3. **Late Data:** Watermark de 2 horas.

Detección de Anomalías: Median Absolute Deviation

Se eligió **MAD** sobre Z-Score por su robustez ante outliers extremos (no asume distribución normal).

```
# Threshold seleccionado: 3.5  
MAD = median(|cost - median(cost)|)  
is_anomaly = (|cost - median(cost)| / (MAD * 1.4826)) > 3.5
```

7. Serving Layer: Cassandra

Diseño Query-First

- **Principio:** Una tabla por patrón de consulta.
- **Denormalización:** Agresiva para evitar joins en lectura.
- **TTL:** 90 días para datos de alto volumen (`finops_daily`).

Tablas Implementadas

1. `finops_daily_usage` (Costos/Anomalías)
2. `org_service_cost_summary` (Top-N Ranking)
3. `support_tickets_summary` (SLA/CSAT)
4. `revenue_monthly` (Finanzas)
5. `genai_usage_daily` (Producto)
6. `carbon_footprint_daily` (Sostenibilidad)

8. Consultas de Demostración

Q1: Costos y Requests Diarios por Org/Servicio (Últimos 30 días)

Análisis de tendencias de costo y detección de anomalías de consumo.

```
SELECT
  org_id,
  usage_date,
  service,
  total_cost_usd,
  total_requests,
  is_anomaly,
  anomaly_score
FROM cloud_analytics.finops_daily_usage
WHERE org_id = '<sample_org>'
      AND usage_date >= date_sub('<snapshot_date>', 30)
ORDER BY usage_date DESC, total_cost_usd DESC
```

Q2: Top-10 Servicios por Costo Acumulado (Ventanas 1d/7d/14d)

Priorización de optimización de costos, identificación de servicios de mayor impacto.

```
SELECT
    org_id,
    service,
    rank_14d,
    cost_14d,
    cost_7d,
    cost_1d
FROM cloud_analytics.org_service_cost_summary
WHERE org_id = '<sample_org>'
      AND summary_date = '<max_date>'
      AND rank_14d <= 10
ORDER BY rank_14d ASC
```

Query 3: Evolución de Tickets de Soporte por Severidad

Monitoreo de calidad de soporte, análisis de SLA compliance y satisfacción del cliente.

```
SELECT
    org_id,
    ticket_date,
    severity,
    total_tickets,
    sla_breaches_count,
    avg_csat
FROM cloud_analytics.support_tickets_summary
WHERE org_id = '<sample_org>'
    AND ticket_date >= date_sub('<snapshot_date>', 30)
ORDER BY ticket_date DESC
```


Query 4: Revenue Mensual con Desglose de Créditos/Impuestos

Análisis financiero, forecasting, cálculo de ARR (Annual Recurring Revenue).

```
SELECT
    org_id,
    billing_month,
    currency,
    total_due_local,
    credits_applied,
    tax_amount,
    net_revenue
FROM cloud_analytics.revenue_monthly
WHERE org_id = '<sample_org>'
ORDER BY billing_month DESC
```

Query 5: Uso de GenAI - Tokens y Costo Diario

Tracking de adopción de servicios GenAI, análisis de eficiencia de uso de tokens.

```
SELECT
    org_id,
    usage_date,
    total_tokens,
    total_requests,
    total_cost_usd,
    avg_tokens_per_request
FROM cloud_analytics.genai_usage_daily
WHERE org_id = '<sample_org>'
    AND usage_date >= date_sub('<snapshot_date>', 30)
ORDER BY usage_date DESC
```

Query 6: Huella de Carbono Diaria (Métrica de Sostenibilidad)

Reporting de sostenibilidad corporativa, cálculo de compensaciones de carbono.

```
SELECT
    org_id,
    usage_date,
    total_carbon_kg,
    events_with_carbon_data,
    ROUND(total_carbon_kg / events_with_carbon_data, 4) as avg_carbon_per_event
FROM cloud_analytics.carbon_footprint_daily
WHERE org_id = '<sample_org>'
    AND usage_date >= date_sub('<snapshot_date>', 30)
ORDER BY usage_date DESC
```

¡Gracias!

Cloud Provider Analytics

Big Data - Proyecto Final